PICCTS

Python Interface Coupling generiC Transport and Speciation

User's Manual

Commissariat à l'Energie Atomique

LAFOND Anatole

anatole.lafond@cea.fr

Contents

1. Introduction

PICCTS (Python Interface Coupling generiC Transport and Speciation, Lafond et al. (2026)) is a user-friendly generic interface using generic modules decoupling the continuity equation governing a thermo-hydro-mechanical-chemical (THMC) system as :

$$\frac{\partial c_i}{\partial t} + \nabla \cdot J_i = \pi_i \tag{1.1}$$

Where c_i, J and π_i are respectively the concentration, flux and source/sink term of species i. This latter equation is further freely decoupled by a certain operator splitting (OS) method, leading to the resolution of its decoupled subparts by user-defined modules. A module is a commercial or freely-distributed external software (e.g., COMSOL or PhreeqC) implemented in PICCTS to apply a certain chemical or transport operator to the system (as defined by Zysset).
The generic communication matrix of PICCTS guarantees full independence and interchangeability of its internal components (i.e., modules and OS). As a flexible coupling framework, PICCTS can facilitate both benchmarking studies (between different modules and OS methods) and the modelling of case-study applications.
Currently, 4 modules (PhreeqC, xGEMS, ORCHESTRA, COMSOL) and 5 OS schemes (additive, alternative, SNIA, Strang and symmetrical) are implemented in PICCTS, freely accessible in REF.
This user-manual intends to help you from your first hands-on experience to the implementation of more advanced, user-defined system couplings, and gently comes alongside two test cases (see Lafond et al. (2026) for further details on results) :

- a classical case of cation exchange of Parkhust and Appelo (2013) coupling PhreeqC-COMSOL and ORCHESTRA-COMSOL.
- a kinetic case of a pesticide infiltration coupling PhreeqC-COMSOL.

Neat segmentation and independence of modules is obtained through a standardized communication matrix. Dedicate operators will further be applied onto this latter matrix. This matrix is taken as both an input and output.

Note : PICCTS have been financed by the EURAD2 HERMES European project and developed during my PhD thesis.
PICCTS being still under development, any feedback, both on the code and this manual, is welcome :)

2. How to run PICCTS

Although other versions may work, PICCTS is guaranteed to run on Python 3.12.8 with the following packages:

Package name	Version
phreeqpy	0.6.0
pandas	2.3.3
numpy	2.2.4
pyORCHESTRA	1.0.1
MPh	1.3.1
xGEMS	2.0.1

Table 2.1: PICCTS' Python package

Note that only the coupled module will import their respective Python package, i.e. coupling ORCHESTRA-COMSOL solely need pyORCHESTRA and MPh to be installed.
To install a missing Python package, please use the "pip install pypackage" command through your Python bash, in which "pypackage" is your missing package. Coupling COMSOL in PICCTS needs COMSOL to be installed externally :

Software	Version
COMSOL	6.2, 6.3, 6.4

Table 2.2: COMSOL required versions

2.1 Test cases

Please download the cation exchange test case (PhreeqC-COMSOL or ORCHESTRA-COMSOL) or the kinetic test case (PhreeqC-COMSOL) accordingly to your installed Python packages. Open cationCase.py or kineticCase.py and change *enginePath* leading to your engine folder, and then run this Python file. Python files cationCase.py and kineticCase.py are called PICCTS input files, as they contain precise keywords defining the system of interest (see 5 for keywords list) and a little block launching PICCTS. All PICCTS keyword in this manual will be presented in *italic*. Note that the .mph file of the test cases is provided, but not the required COMSOL licence.

2.2 Output

Running on of these test case will create several folders, each of them containing communication matrixes under .txt format. These communication matrixes are reported at every time step of the coupling, in which the number of the time-step is reported at the end of each communication file name. The different communication matrixes are reported in the following folders:

- outputHistory gathers communication matrixes after (a potential) processing of the chosen OS, at the end of the time-step.
- outputTransport gathers communication matrixes after transport solver.
- outputSpeciation gathers communication matrixes after speciation solver.
- VTUdata gathers communication matrixes under .vtu format after COMSOL processing.

As an example, 'PhreeqC14.txt' is the communication matrix of the 14^{th} time-step after PhreeqC speciation, reported under the outputSpeciation folder.

2.3 Pre-processing

Post-processing the different communication matrixes is generally necessary to get the data of interest. You have two ways to do so:

- via Paraview using the .vtu communication matrixes reported in the VTUdata folder. This is a convenient method, but please remind that only the transported communication matrixes (i.e., after COMSOL step) are written under .vtu format.
- via a direct processing of .txt communication matrixes. To do so, PICCTS comes natively with a little Python code for 1D systems (called postProcess.py). It will display concentration profiles and breakthrough concentrations of selected species, at a defined time-step. Note that all the communication matrixes are standardized under the same framework, which makes the post-processing much easier !

2.4 Defining your system

System definition is mainly carried by the PICCTS input file. Notably, PICCTS couples internal (PhreeqC, ORCHESTRA, xGEMS) and external (COMSOL) modules. The latter kind of modules are already fully defined using PICCTS keywords, whereas former ones need some pre-processing to be coupled. The general guidelines for pre-processing are module-dependent and further described in 3. Note that coupled modules do not lose any modelling capacity with respect to their stand-alone equivalent

3. PICCTS modules

This chapter will detail the implemented modules in PICCTS. For the sake of simplicity, we will only consider a reactive transport with no cross dependencies, i.e., assuming species as only coupled variables. Cross-dependencies in PICCTS are detailed in section 4. Chosen PICCTS module are wrapped in Python using Python package (see associated Python package table 2.1). A correct installation of the chosen Python packages is assumed. Below each subsection presenting a module, you will find a link referring to its associated Python package distribution (at the date of 26/04/2026).

3.1 COMSOL

Definition of COMSOL is not carried by PICCTS and should be externally pre-processed before coupling with PICCTS, where minimal information is transferred to the .mph file through the Python Java-wrapper package MPh. It can be done via its JAVA functions or through the COMSOL GUI. A COMSOL model needs to be built accordingly to the three keys parameters listed below. It is assumed that the user has a certain experience with the COMSOL GUI, so that the different features of the Model Builder are self-explanatory. Note that the COMSOL Model Builder features are written below in **bold**.

- First, **Interpolation Function** should be in "File" mode, using the first columns as spatial coordinates while the others columns refer to coupled species. Note that the order of the species carried by **Interpolation Function** should display the exact same order referred by *systemSpeciation*.
- In a dedicated interface (e.g., Transport of Diluted Species for modelling Fick's law), make sure that initial concentration of species refers to its **Interpolation Function** counterpart. The remaining definition of the interface is left to the user. Please note that the interface and **Interpolation Function** have no need to contain the same number of species. In the case where fixed species exist in the system, there is no need to include them in the transport interface.
- Last, **data** should solely display the *systemSpeciation* order. The user can save images of the transported system after each time-step by using the keyword *comsolData*. If fixed species are present in the system, please insert their initial concentration reported by the **Interpolation Function**.

The two test cases strictly follow these guidelines. Note that the kinetic test case contains pressure as supplementary coupled parameters. See section 4 for a complete

list of available coupled parameters and the different ways to implement them.
COMSOL is natively parallelized.

3.2 PhreeqC

PhreeqC is a mass action law speciation solver (Parkhust and Appelo (2013)) further wrapped in Python.
Chemical system definition carried by PhreeqC is fully internalised by PICCTS thanks to various keywords (see section 5).
However, extreme pH and/or pe conditions far from equilibrium will occur when equilibration have been set using H^+ nor OH^- as master species. A precise knowledge is still necessary to fully model acidic and redox plumes in a coupled system. To circumvent this effect, PICCTS carries two methods to model acidic plumes using PhreeqC.

3.2.1 Acidic transport

Echoing H^+ and OH^- species Taking $C^t_{A^z}$ as the transported non-equilibrated nodewise concentration at time t of A^z, with A^z being H^+ or OH^-, and $C^{t-dt}_{A^z}$ as the equilibrated nodewise concentration at time $t - dt$, if $C^t_{A^z} - C^{t-dt}_{A^z}$ is positive, the following heterogenous reaction is considered:

$$\equiv A^z + \text{IonPhantom}^z \rightleftharpoons \equiv \text{IonPhantom}^z + A^z$$

Where $\equiv$ is an ion exchange site, and IonPhantom^z an ion carrying the same charge z of A^z, at an initial concentration of $[\text{IonPhantom}^z] = C^t_{A^n} - C^{t-dt}_{A^n}$ mol $\cdot$ kgw^{-1}.
If $C^t_{A^n} - C^{t-dt}_{A^n}$ is negative, the reverse reaction is considered:

$$\equiv \text{IonPhantom} + A \rightleftharpoons \equiv A + \text{IonPhantom}$$

With $[\equiv\text{IonPhantom}] = C^{t-dt}_{A^n} - C^t_{A^n}$ mol $\cdot$ kgw^{-1}.
This little trick, much similar to fixing the pH using an H^+ pseudo-phase, allows the transport of H^+ and OH^- species.
Initial pH value at time t is taken as the equilibrated pH at $t - dt$. However, this method artificially increases the ionic strength (IonPhantom being a charged species) and thus may over-estimate the deviation from ideality, especially when large acidic/basic plumes are considered. In a case where no convergence is reached, due to a too large acidic/basic plume, reducing the time-step and choosing appropriate geometrical dependent counter charge species seems unavoidable. The IonPhantom species is not further reported by the communication matrix. Thus, the reported equilibrated solution may not be charge-balanced. One shall use this method **solely** when the method of the following paragraph does not converge by any means. This method does not provide any guarantee of accuracy.

SOLUTION_MODIFY keyword This method uses the PhreeqC keyword SOLUTION_MODIFY, which allows the user to define the precise amount of H and O nodewise, considering nodes are electrically neutral (although electrical charge may be a coupled parameters as well). This assumption may no longer hold when modelling

a double layer, for example. SOLUTION_MODIFY keyword of PhreeqC is accessible through the **solMod** keywords of PICCTS, assuming a molal concentration of 55.508 mol kg_w^{-1} if species 'H_2O' is not found in the communication matrix.
This method needs a precise knowledge of non-aqueous H and O to precisely set the correct redox and pH at equilibrium.

3.2.2 Redox transport

Modelling redox plumes in a similar way to the first pH transport approach assume that pH and pe would carry the same physicochemical information (Hostettler (1984); Thorstenson (1984)). Strict modelling of redox plume is solely reachable through the definition of total nodewise moles of H and O, via the SOLUTION_MODIFY PhreeqC keyword. More generally, redox plume is solely can be solely modelled though transport of redox species.

3.3 ORCHESTRA

ORCHESTRA (Meeussen (2003)) is an object-oriented solver in which any equations carried out may be accessible by the user, unlike other standard chemical equilibrium solver (e.g., PhreeqC, CHESS, MINTEQ) where equations (although accessible) are hard-coded directly within the code. ORCHESTRA solver was first coded in JAVA, but have been translated in C++ and recently wrapped in Python, which was finally implemented in PICCTS. As the most of chemical information is carried via the .inp file, rather few keywords are implemented in PICCTS for ORCHESTRA. Notably, only the 'attributes ' of species need to be somewhat provided (e.g. Na.solid if Na is a phase species). Please refer to the .inp file and its imported 'concert.txt' file for further explanations on species attributes.

3.4 xGEMS

xGEMS is a speciation solver minimizing the Gibbs free energy to reach the equilibrium state, while most of others speciation solvers (PhreeqC, CHESS) use the action mass law. Although these two thermodynamical variables are of course linked and must theoretically give the same equilibrium, xGEMS speciation may found an advantage on complex, non-ideal system at high solid-solution ratio (REF). Python-wrapped xGEMS is implemented in PICCTS. xGEMS equilibrates independent components to dependent components, whereas PhreeqC and ORCHESTRA work with primary and secondary species. As PICCTS already includes a decomposition method to access trivial secondary species onto its primary species (e.g., $Na_2SO_4 \rightarrow$ 1 Na, 4 S and 4 O), non trivial decomposition are not accessible with PICCTS (e.g., 'Calcite' into 1 C, 1 Ca and 3 O) as PICCTS does not (currently) retrieve the stoichiometry matrix from the xGEMS database. The xGEMS keyword *nonTrivialDC* fills this gap.

3.5 Further development

- Instantiation of the COMSOL client only once at the beginning of the PICCTS run, instead of once every time-step. Will spare around $\sim$ 1 sec per step ...

- Implementation of all PhreeqC and xGEMS cross-dependencies. A complete list of cross-dependencies reachable by PhreeqC and xGEMS is reported section 4.
- Implementation of a generic keyword for ORCHESTRA assessing for cross-dependencies. Unlike PhreeqC or xGEMS, the nature of potential cross-dependencies is not fixed by a dedicate keyword in the PhreeqC-Python or xGEMS-Python wrapper, but rather set by the .inp and associate files.
- Outer-parallelization (e.g., running PhreeqC, ORCHESTRA and xGEMS within the same time-step using identical initial conditions) is to be implemented in PICCTS. Will be useful for error estimation and adaptative time-stepping.

Please let me know if you would like to see any additional development !

4. Cross-dependencies

Is called a cross-dependency variable any variable (outside of nodewise speciation), taken as an input and an output of the chosen modules. A cross-dependency may be partial (i.e., not necessary for all the modules, as the pressure in the kinetic test case) or complete (all modules impacts and are impacted the chosen cross-dependency). The range of cross-dependencies available is directly linked to the modules chosen. Below is listed, for each module, the different cross-dependencies **that will be** available. As PICCTS is still under development, they are not implemented. Please let me know which one(s) you would like to see in priority. Setting up a new cross-dependency will take me less than 1h ...

Table 4.1: Cross-dependencies with respect to modules.

Module	Cross-dependency variables
PhreeqC	Temperature, conductance, density, viscosity (solely computed with respect to pure water, which depends on temperature), ionic strength, dielectric constant, volumes of gas, volume aqueous phases, water compressibility, osmotic coefficient, pressure and fugacity, pH/pe, diffuse layer composition.
xGems	temperature, volume, mass, ionic strength, pressure, heat capacity, density.
ORCHESTRA	Cross-dependencies should be defined by the .inp and associate files.
COMSOL	Cross-dependencies should be defined during the pre-processing of COMSOL and further acknowledged within the PICCTS interface.

Uncoupled variables reported in user-defined communication matrixes are the same as the coupled cross-dependencies variables. See subsection 2.2 for further details on user-defined communication matrixes.

5. Keywords

Available PICCTS keywords are listed alphabetically 5.1, including a description, default value and related module(s) as dependency. A keyword may be related to one or several modules (i.e., PhreeqC, xGEMS, ORCHESTRA, COMSOL), whereas 'None' value refers to a module-independent keyword (e.g., keyword *timeUnit* refers to the unit of the time discretization, is not module-dependent). These keywords should be reported in the PICCTS input file (see section 2 for detail on the PICCTS input file).

There is currently 42 different keywords available, but I suppose that only 5 to 10 keywords will be commonly used. I have explicitly included them in the test cases.

Keyword	Description	Default value	Type	Dependency
chemModule	Speciation module choice. 1 for PhreeqC, 2 for xGEMS, 3 for ORCHESTRA	1	Integer	None
comsolCore	Number of cores for COMSOL parallellization. With None value, COMSOL automatically choses the number of cores.	None	Integer	COMSOL
comsolCouplingTags	List of COMSOL tags organized in the following order : component tag, interpolation function tag, study tag, communication matrix exporting tag.	['comp1', 'int1', 'std1', 'data1']	List of strings	COMSOL
comsolData	COMSOL tag related to the user-defined communication matrix. A list of tags may be included as well, in which all tages will be further exported as user-defined communication matrix.	'data1'	String or list of strings, e.g. 'data1' or ['data1', 'data2 '].	COMSOL
comsolVTU	Export user-defined communication matrix as .vtu. Will not overwrite the standard exportation in .txt format. Large number of time steps saved under .vtu format may occupy a significant amount of disk space. Every time step is saved.	True	Boolean	COMSOL
constantSpecies	Imposes a constant molality of chosen primary species. Constant molality of H and O primary species will probably result in extreme pH and/or pe conditions.	{ }	Dictionary where primary species are keys and constant molalities are values, e.g. { 'H': 111.123456789}	xGEMS

crossDep	Potential cross dependencies. They are assumed to be as complete as possible (see section 4 for the partiality of cross-dependencies). **This keyword will be soon implemented. Does not currently work.**	None	Dictionary where boolean cross-dependencies are keys and boolean variables as values, whereas non-boolean cross-dependencies take a list of strings as values,e.g. {boolCrossDep : True, listCrossDep : ['sp1','sp2']}	PhreeqC, xGEMS
cutoff	Concentration threshold of species. Assumed to be absent from the considered node if its attributed concentration if inferior.	10^{-16}	Integer / float	ORCHESTRA (xGEMS to do)
cutoffAq	Concentration threshold of aqueous species. Assumed to be absent from the considered node if its attributed concentration if inferior.	10^{-20}	Integer / float	PhreeqC
cutoffEch	Concentration threshold of ion exchange species. Assumed to be absent from the considered node if its attributed concentration if inferior.	10^{-20}	Integer / float	PhreeqC
cutoffPha	Concentration threshold of surface species. Assumed to be absent from the considered node if its attributed concentration if inferior. Note that a positive threshold will prevent the phase to precipitate when completely solubilized in the system.	-10^{-99}	Integer / float	PhreeqC
cutoffSurf	Concentration threshold of phase species. Assumed to be absent from the considered node if its attributed concentration if inferior.	10^{-20}	Integer / float	PhreeqC
edl	Presence of an eletrical double layer when a surface species is present in the system. Mass of all surface species is assumed at 1 g at 100 m^2 g^{-1}.	True	Boolean	PhreeqC

firstStepEquilibrium	Possibility to equilibrate the initial conditions before the coupling.	False	Boolean	PhreeqC, xGEMS and ORCHESTRA
fixpH	Will fix equilibrated pH, considering an imaginary solid phase of H^+.	None	Integer / float	PhreeqC
geometry	1 for 1D, 2 for 2D and 3 for 3D.	1	Integer	None
independentComponents	Independent components. The order of the independent components should exactly match the order of the associated xGEMS database.	[ ]	List of strings	xGEMS
initialConditions	Initial conditions path. Accepted format is .txt.	'ic.txt'	String	None
kinetics	Presence of kinetic reactions in the considered system.	False	Boolean	PhreeqC
mineralReversibility	Mineral reversibility of the phases. Only accepted values are '-d' or '-dissolve_only' for dissolution only and '-p' or '-precipitate_only' for precipitation only.	None	Dictionary where phase species are keys and reversibility keywords are values, e.g. {'Calcite ' : '-d'}	PhreeqC
nonTrivialDC	Stoechiometry matrix under PICCTS formalism. Only 'non-trivial' decomposition of dependent components (e.g., Calcite, Dis-Dolo, etc.) should be reported, whereas the decomposition of trivial dependent components (e.g., Na_2SO_4, H_3Edta^-) is already implemented in PICCTS. Note that any independent components reported in *nonTrivialDC* should have been reported in *primarySpecies* as well.	{ }	Dictionary where dependent components are keys and values are dictionaries, in which independent components are keys and stoichiometry coefficient are values, e.g. { 'Calcite': {'Ca': 1, 'C': 1, 'O': 3 }}	xGEMS
operatorSplitting	Operator splitting choice. 1 for SNIA, 2 for Strang splitting, 3 for alternative splitting, 4 for additive splitting, 5 for symmetrical splitting.	1	Integer	None
pHdefault	Default initial pH value.	7	Integer / float	PhreeqC
PIDnbr	PID number for parallellization.	1	Integer	PhreeqC, xGEMS and ORCHESTRA
primarySpeciesAq	Aqueous primary species.	[ ]	List of strings	PhreeqC

primarySpeciesPha	Phase species.	[ ]	List of strings	PhreeqC
primarySpeciesSurf	Surface primary species.	[ ]	List of strings	PhreeqC
primarySpeciesEchSorbed	ORCHESTRA attribute related to ion exchange species.	None	Dictionnary where attributes are keys and list of species are values, e.g. {attribute : ['sp1', 'sp2'] where attribute is an attribute figuring in the '.inp' file (e.g., 'solid' for solid phase species.)	ORCHESTRA
primarySpeciesPhantom	Decomposed primary species, but not taken into account in equilibrium.	['O', 'H', 'H_2O', 'pH', 'pe']	List of strings	PhreeqC
SI	Saturation indices of phases.	Saturation indice of 0 for all phases	Dictionary where phases are keys and saturation indices are values, e.g. {'Calcite ': 1 }	PhreeqC
solMod	Equilibrium using the PhreeqC SOLUTION_MODIFY keyword. Overwrites *acidicTrspt* and *pHdefault* keywords.	True	Boolean	PhreeqC
speciationEch	Ion exchange species. Secondary species is expected (e.g., ['X_Na'] instead of ['X_'] for a sodic ion exchange site).	[ ]	List of strings	PhreeqC
speciationCharge	Potential presence of a counter-charge species.	None	Boolean	PhreeqC
speciesCharge	Counter-charge primary species. They are tested in the order, in the whole system. Any of ['H', 'ph', 'pH'] set the pH to charge-balance the system.	['pH']	List of strings	PhreeqC
speciesChargeGeometry	Geometrical dependent counter-charge primary species. They are tested in the order, in the whole system. Overwrites *speciesCharge*.	None	List of strings	PhreeqC

stepReprise	Imposes the step number at which the run begins. Ex. : previous run stopped at step n°5 due to a too large time step (e.g., 1h). Using this keyword you may continue this simulation starting at step n°5 using a 30 minutes time step. If simulation succeeds, files will be written accordingly to the reprise step number (n°6 in this case). One should use stepReprise = 5, and change the time discretization so that the 6th step has a reduced time step. Make sure that *InitialConditions* contains the right initial conditions file (e.g, last communication matrix written in outputHistory folder).	False	Integer	None
systemSpeciation	System speciation including phase, solution, ion exchange, isotopes, surface species. Associated modules should present the same species order. The following names are forbidden: X,Y,Z,x,y,z.		List of strings	None
timeUnit	Time unit. 's' for seconds. 'min' for minutes. 'd' for days.	's'	String	None
TrsptAcide	Acid and base transport considering the H^+ and OH^- echoing method. Overwrites *pHdefault*.	False	Boolean	PhreeqC
trsptModule	Transport module choice. 1 for COMSOL.	1	Integer	None
userVariablesBool	User-defined boolean variables written in the communication matrix user. Accepted variables are: alkalinity, charge_balance, ionic_strength, pe, pH, percent_error, reaction, temperature, water. Please see lines 10 to 18 of exercise 11 of Parkhust and Appelo (2013) to check the different boolean values.	{ }	Dictionary where PhreeqC keywords are keys and boolean values are values, e.g. {'alkalinity': True}	to be done: PhreeqC, xGEMS

userVariablesList	User-defined list variables written in the communication matrix user. Accepted variables are: activities, calculatedValues, gases, isotopes, kineticReactant, saturationIndice, solidSolution, totals. Please see lines 19 to 29 of exercise 11 of Parkhust and Appelo (2013) to check the different list values. Note that, by default, molalities of every species (i.e., solution, surface, ion exchange, and phase species) are written in the communication matrix user. Forbidden PhreeqC keywords are 'molalities' and 'equilibrium_phases'.	{ }	Dictionary where PhreeqC keywords are keys and list of species are values, e.g. {'activities': ['H_2O']}	to be done: PhreeqC, xGEMS
water	Mass (kg) of nodewise water. Correcting the phase molality with respect to 1 kg of water may work as well.	1	Integer / float	PhreeqC

Table 5.1: Keyword list.

6. Troubleshooting

Error may occur in a PICCTS run. They are seperated in two categories: explicit (returned in the Python interpreter and reported in the warning.log file) and implicit (not returned by any means).
Note that the warning.log file reports every explicit error (fatal or not) occurring within a run, with clear descritption of the component where the error emerges.
You will find below a list of errors, classed alphabetically.

6.1 Explicit errors

BrokenProcessPool: A process in the process pool was terminated abruptly while the future was running or pending.
→Please reboot your Python kernel.

com.comsol.util.exceptions.FlException: Cannot open native file.
Messages: Cannot open native file.
→May be due to insufficant disk space. I got this issue several times even if I got >50Go available. I suspect MPh to be behind this error. Does not occur when COMSOL alone is ran. Please free some space.

com.comsol.util.exceptions.FlException: Cannot create native file.
Messages: Cannot create native file.
→May be due to insufficant disk space. I got this issue several times even if I got >50Go available. I suspect MPh to be behind this error. Does not occur when COMSOL alone is ran. Please free some space.

com.comsol.util.exceptions.LicenseException: Could not obtain license for 'Nernst-Planck Equations (npe)'.
Required product: Chemical Reaction Engineering Module.
Messages: Could not obtain license for 'Nernst-Planck Equations (npe)'.
Required product: Chemical Reaction Engineering Module.
License error: -4.
Licensed number of users already reached.
→A colleague is probably using the required licence (NPE, in this case). Please threaten him to free the licence. They may be seen at (Windows OS using COMSOL v.6.3, may require admin rights) :
`C:\{COMSOL path}\COMSOL\COMSOL63\Multiphysics\license\win64\`

`lmutil lmstat -a -c ../license.dat`

com.comsol.util.exceptions.FlException: Not connected to a server.
Messages: Not connected to a server
Waiting for a licence ?
→Make sure you are connected with Internet, reboot your Python kernel.

com.comsol.util.exceptions.FlException: The following feature has encountered a problem:
Messages:
The following feature has encountered a problem:
- Feature: Dependent Variables 1 (sol1/v1)
Undefined variable.
- Variable: Clminusi
- Geometry: geom1
- Domain: 1
Failed to evaluate expression.
- Expression: Clminusi
→Please check that the variable OHmoinsi is implemented in the Interpolation Function. This issue may arise when updating COMSOL version from 6.3 to 6.4.

FileNotFoundError: [Errno 2] No such file or directory: `'{some path}\engine.py'`
→Make sure that *enginePath* keyword contains the right path.

FileNotFoundError: [Errno 2] No such file or directory: "store.txt"
→Please launch the PICCTS input file (see section 2).

RuntimeError: Could not find a supported Comsol installation.
→Make sure Comsol ("../comsol63/bin" using Comsol 6.3) is in your working path. Reboot your PC. If you are on Linux or MacOs, maybe this webpage could help you. Because this issue is quite common in MPh version ≤1.1.4 as seen in these github resolutions, make sure MPh is up-to-date. As PICCTS have never been launched in MacOS environment, please consider using PICCTS on Windows or Linux.

RunString: No database is loaded
→Make sure *chemPath* is correct. If so, this error refers for an invalid database as well. Please run PhreeqC alone (i.e., not in PICCTS), to surely determine the mistake in your database.

ValueError: max_workers must be ≤ x
→Looks like you reached your maximum number of parallelized PID ! Please decrease the number under this limit using the *PIDnbr* keyword.

6.2 Implicit errors

Transport part does not seem to take into account the speciation.
→Make sure the **Interpolation Function** is indeed linked to the initial conditions

of your system. See 3.1 for further details on pre-processing the COMSOL modules accordingly to PICCTS communication framework. Be sure the **Initial conditions** node referring the **Interpolation function** is indeed activated and not dominated by another **Initial conditions** interface.

Bibliography

John D. Hostettler. ELECTRODE ELECTRONS, AQUEOUS ELECTRONS AND REDOX POTENTIALS IN NATURAL WATER. 1984.

Anatole Lafond, Clément Lynde, and Romain V. H. Dagnelie. Piccts : an open-access python interface coupling generic transport and speciation solvers for thmc modelling. *Under submission ..*, vol(nb):pg1–pg2, 2026.

Johannes C. L. Meeussen. ORCHESTRA: An Object-Oriented Framework for Implementing Chemical Equilibrium Models. *Environmental Science & Technology*, 37(6): 1175–1182, March 2003. ISSN 0013-936X, 1520-5851. doi: 10.1021/es025597s. URL https://pubs.acs.org/doi/10.1021/es025597s.

D. L. Parkhust and C.A.J Appelo. A computer program for speciation, batch-reaction, one-dimensional transport, and inverse geochemical calculations. *U.S Geological Survey, Denver, Colorado*, 2013.

Donald C. Thorstenson. THE CONCEPT OF ELECTRON ACTIVITY AND ITS RELATION TO REDOX POTENTIALS IN AQUEOUS GEOCHEMICAL SYSTEMS. 1984. Series: Open-File Report.